

YouTubeAPIManager

get_instance()

- Retorna a instancia da classe YouTubeAPIManager (singleton)

get_new_youtube_client()

- Recupera as chaves definidas no arquivo de configurações
- Se já tentou todas as chaves:
 - Espera até as 5 da manhã
 - Defini a chave de index 0
- se não:
 - define a proxima chave

make_api_request()

- Realiza uma requisição na API do YouTube

Código

main()

- Recupera as queries do arquivo config
- Cria os diretórios para armazenar as saídas
- Atualiza o estado global "status" para "working"
- Captura a data atual do arquivo "atual_date.csv"
- Cria os datetime start_date e end_date
- Recupera REGION_COED e RELEVANCE_LANGUAGE do arquivo de configurações
- Cria um conjunto para armazenar os ids já processados e recupera esses dados do arquivo "processed_videos.csv"
- Inicia o envio de dados para a api de status (opcional)
- gera uma lista de intervalos que contem tuplas (inicio, fim)
- Para cada intervalo:
 - Atualiza o arquivo "atual_date.csv" com a data atual da coleta
 - pada cada query:
 - Atualiza os status globais atual_query e query_progress
 - define as datas de inicio e fim da pesquisa com base no for de intervalos
 - Realiza uma pesquisa na API do YouTube **[make_search_request()]**
 - Captura os videos retornados na pesquisa
 - Se não retornou nenhum vídeo:
 - Vai para a proxima query
 - **continue**
 - Para cada vídeo:
 - Recupera o id do vídeo
 - Se o id do vídeo não está no conjunto de processados:
 - Captura os detalhes do vídeo **[get_video_details()]**
 - Extrai o número de comentários dos detalhes do vídeo
 - Se possui comentários:
 - Processa o vídeo **[process_video()]**

get_video_details()

- Recupera a instancia da **[YouTubeAPIManager]**
- Realiza uma requisição na API com os parâmetros definidos
- Extrai os dados importantes
- Monta um dicionário com os dados do vídeo
- Retorna os dados do vídeo

get_channel_details()

- Recupera a instancia da **[YouTubeAPIManager]**
- Realiza uma requisição na API com os parâmetros definidos
- Extrai os dados importantes
- Monta um dicionário com os dados do canal
- Retorna os dados do canal

make_search_request()

- Recupera a instancia da **[YouTubeAPIManager]**
- Realiza uma requisição na API do YouTube do tipo pesquisa passando os parâmetros e salva em search_response.
- **return** search_response

process_video()

- Recupera os detalhes do vídeo **[get_video_details()]**

- Salva a quantidade de comentários
- Se a quantidade de comentários está dentro dos limites:
 - salva os dados do vídeo no arquivo "videos_info.csv"
 - Recupera os detalhes do canal `[get_channel_details()]`
 - Salva os dados do canal no arquivo "channels_info.csv"
 - Captura os comentários e replies do vídeo
 - salva os comentários em comments_info.csv `[get_comments()]`

get_comments()

- Recupera a instancia da `[YouTubeAPIManager]`
- cria uma lista para salvar os dados (comments)
- Cria uma variável para salvar o token para a proxima página de comentários
- Enquanto houver token para uma proxima página:
 - Realiza uma chamada na API passando os parâmetros necessários junto do token da lista de comentários
 - Para cada comentário retornado:
 - Recupera os dados do comentário
 - Salva os dados na lista comments
 - Se o comentário tem replies:
 - Recupera a lista de replies do comentário `[get_replies()]`
 - Adiciona os replies na lista comments
 - Atualiza o token de proxima página
 - Se não tiver token:
 - Finaliza o while
 - **break**
- **return** comments

get_replies()

- Recupera a instancia da `[YouTubeAPIManager]`
- cria uma lista para salvar os dados (replies)
- Cria uma variável para salvar o token para a proxima página de replies
- Enquanto houver token para uma proxima página:
 - Realiza uma chamada na API passando os parâmetros necessários junto do token da lista de replies
 - Para cada reply retornado:
 - Recupera os dados do reply
 - Salva os dados na lista replies
 - Atualiza o token de proxima página
 - Se não tiver token:
 - Finaliza o while
 - **break**
- **return** replies